HYPERPARAMETER TUNING, BATCH NORMALIZATION

Outline

- Hyperparameter tuning
  - Tuning process
  - Using an appropriate scale to pick hyperparameters
  - Hyperparameters tuning in practice
- Batch normalization
  - Normalizing activations in a network
  - Fitting batch norm into a neural network
  - Why does batch norm work?
  - Batch norm at test time
- Multi-class classification
  - Softmax regression
  - Training a softmax classifier

- Introduction to deep learning frameworks
  - Deep learning frameworks
  - TensorFlow



deeplearning.ai

Hyperparameter tuning

Tuning process

- There are many hyperparameters in a neural network that need to be tuned during training.

Hyperparameters

α: learning rate → most important hyperparameter to tune.

β: momentum parameter → ~0.9 works well

$\beta_1, \beta_2, \varepsilon$: adam optimization → use $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$

#layers

hidden units

learning rate decay

mini-batch size → is also important

Red: most important

Yellow: second in importance

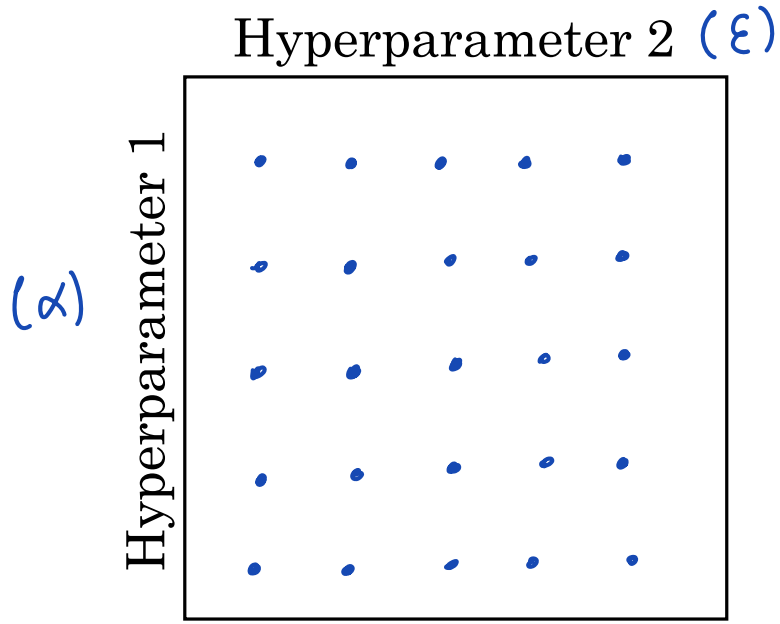
Green: next in importance

Andrew Ng

- How do you select the set of values to explore when tuning a hyperparameter?

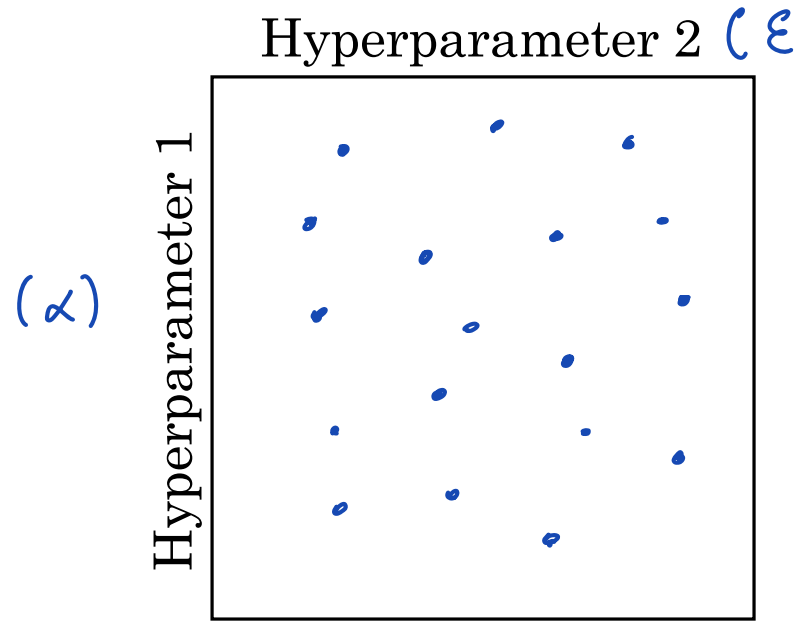
Try random values: Don't use a grid

Earlier machine learning algorithms:



- Sample the points on a grid and systematically explore the values (25 pts).
- If the choice of ε does not make much difference, we have only tried 5 α values.

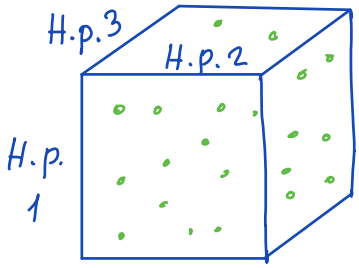
In deep learning:



- Choose the 25 pts. randomly.
- We have tried 25 different α values, and we are more likely to find an α that works really well.

Andrew Ng

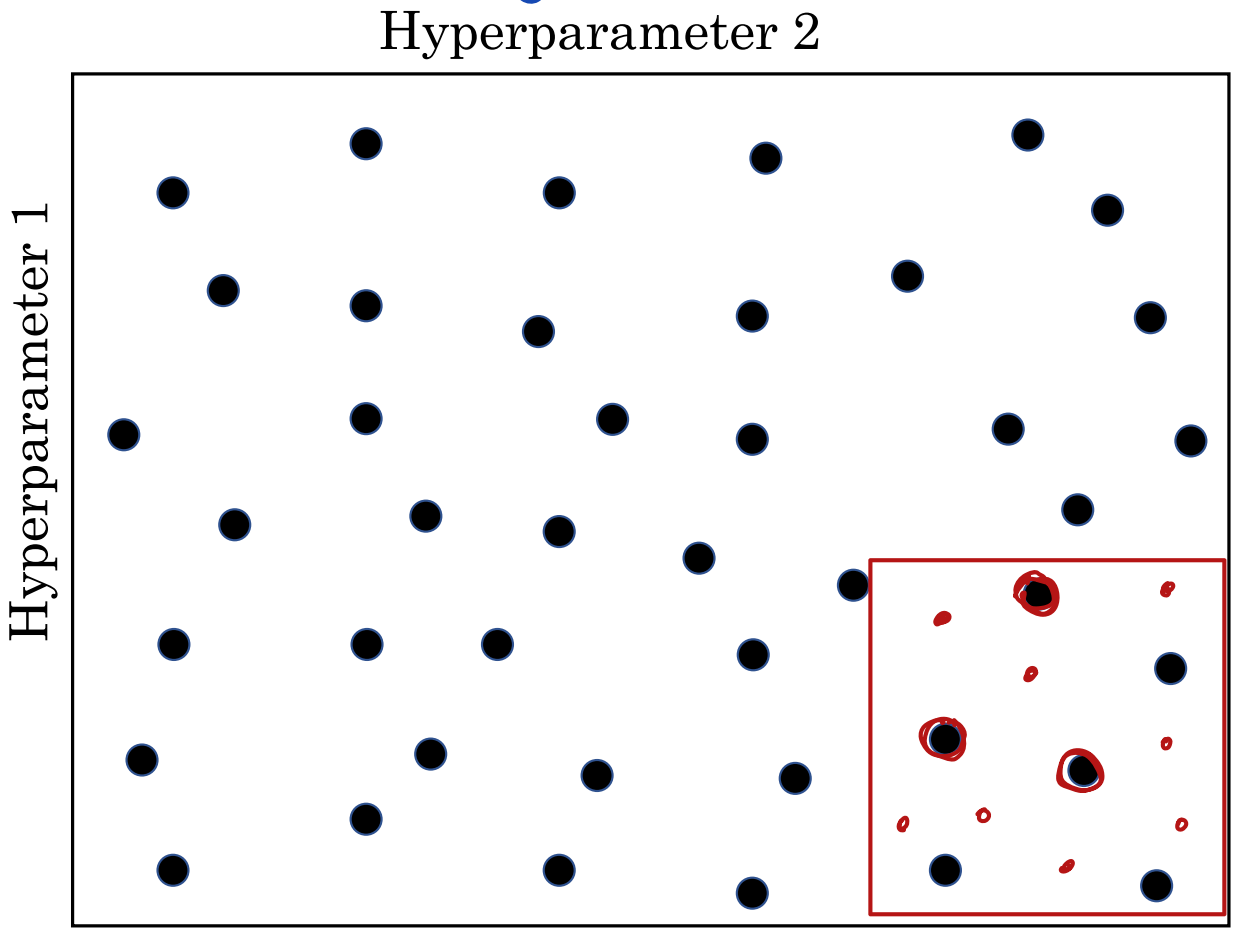
- In practice we are searching more hyperparameters: e.g. 3



- If is hard to know in advance which hyperparameters are really important.

- By sampling at random rather than the grid, we more richly explore the set of possible values for the most important hyperparameters.

Coarse to fine sampling scheme



- Sample the space randomly at first (big black dots)
- Zoom in to the region that works best to sample more densely at random (red points inside the red square). Pick the best point.

Andrew Ng



deeplearning.ai

Hyperparameter tuning

Using an appropriate scale to pick hyperparameters

Picking hyperparameters at random

- Sampling at random doesn't mean sampling uniformly at random, over the range of valid values.
- Instead, it is important to pick the appropriate scale on which to explore the hyperparameters.

e.g. If you want to choose the # of units per layer: $n^{[l]}$ and you think the range of $n^{[l]}$ should be:

- $n^{[l]} = 50, \ldots, 100$



: test random values in this range.

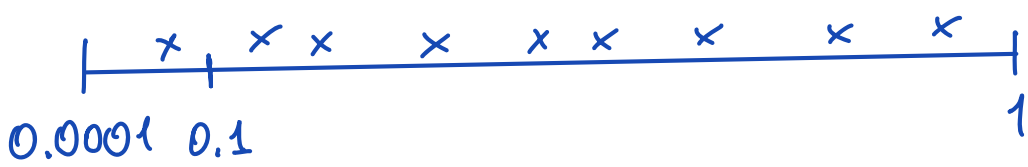
- #layers L: 2-4

  may sample uniformly and try 2-3-4 layers

Andrew Ng

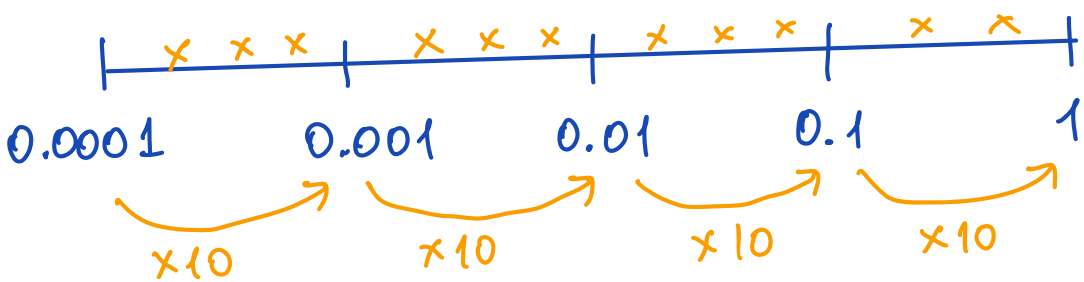
Appropriate scale for hyperparameters

$\alpha = 0.0001, \ldots, 1$ want to search for α in this range.



- If you sample uniformly over the range, 90% of the sampler will be between 0.1 and 1.
- Only 10% of sampler will be between 0.0001 and 0.1.

- It is better to search for hyperparameters on a log scale.
  - Sample uniformly on this logarithmic scale.



Andrew Ng

Implementation in Python:

```
r = -4 * np.random.rand()      # r ∈ [-4, 0]
α = 10^r                       # α ∈ [10^-4, 10^0]
```

- In general we are sampling between 10^a and 10^b on the log scale.
  for sampling between 0.0001 and 1:

  $10^a = 0.0001$ $\qquad$ $b = 1 = 10^0$

  $a = \log_{10} 0.0001 = -4$ $\qquad$ $b = \log_{10} 1 = 0$

- Sample uniformly at random between a and b:

  $r \in [a, b]$

  $[-4, 0]$

  Then, hyperparameter is $\alpha = 10^r$

Hyperparameters for exponentially weighted averages

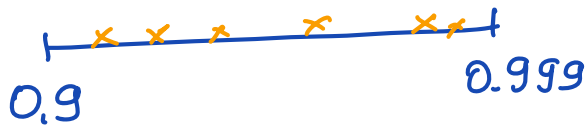
- A tricky case is when searching for the parameter β:

Let's say you search for β between 0.9 – 0.999.

$\beta = 0.9 \ldots 0.999$

like averaging over last 10 days

like averaging over last 1000 days values.



It is not reasonable to sample the range uniformly.

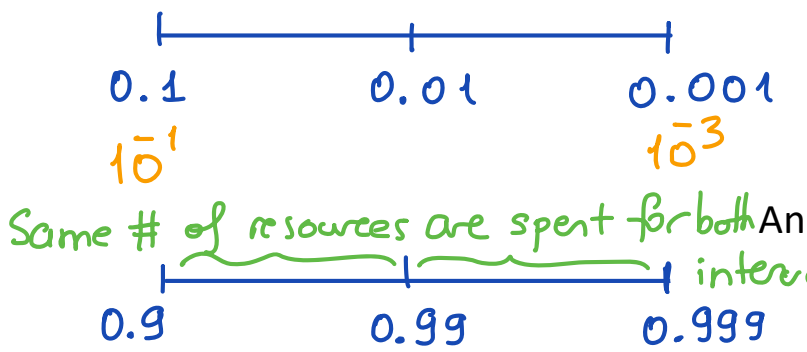
Instead look at $1-\beta$:

$1-\beta = 0.1 \ldots 0.001$

Sample r uniformly btwn $[-3,-1]$

$r \in [-3,-1] \Rightarrow 1-\beta = 10^r$

and $\beta = 1-10^r$



Andrew Ng

- More explanation on why it is a bad idea to sample on a linear scale?
  - When $\beta \approx 1$, the sensitivity of the results you get changes, even with very small changes in β.
    - If β goes from 0.9 to 0.9005 → hardly any change in results
      both are roughly averaging over 10 values
    - If β goes from 0.999 to 0.9995 → huge impact
      averaging over: 1000 values , 2000 values
  - In the formula we have $\frac{1}{1-\beta}$, so it is sensitive to small changes in β, when β is close to 1.
  - By using logarithmic scale we sample more densely in the region where β is close to 1.

- Note: Even if you sample uniformly on a range, you can use a coarse-to-fine strategy to focus on the most useful range of hyperparameter values to sample.

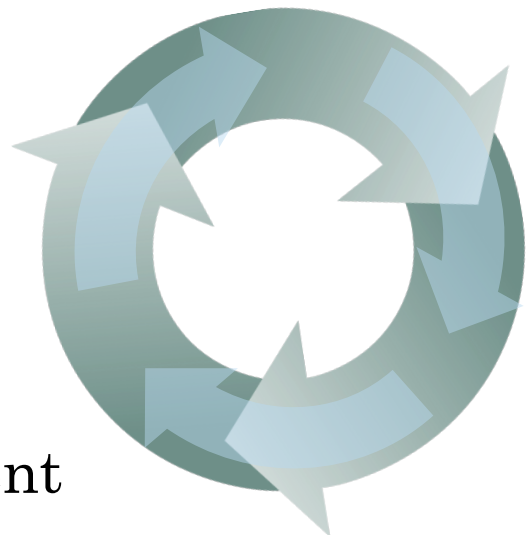


deeplearning.ai

Hyperparameters tuning

Hyperparameters tuning in practice: Pandas vs. Caviar

Re-test hyperparameters occasionally



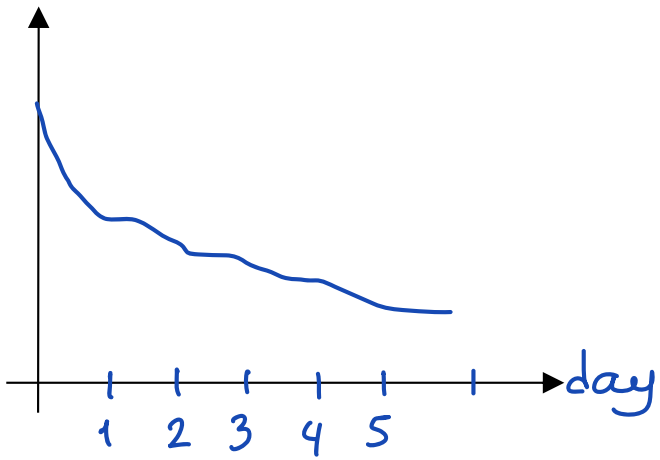
- Ideas in deep learning developed for one domain can be useful in other domains.
  - NLP, Vision, Speech, Ads, logistics, ...
  - Intuitions do get stale. Re-evaluate occasionally.
    - Data may change gradually in time.
    - Evaluate the hyperparameters regularly to check if they're still the best.

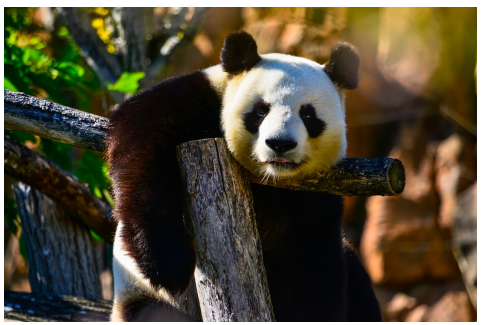
Andrew Ng

- There are two major methods of searching for hyperparameters.

Method 1: e.g. limited GPU resources

Babysitting one model





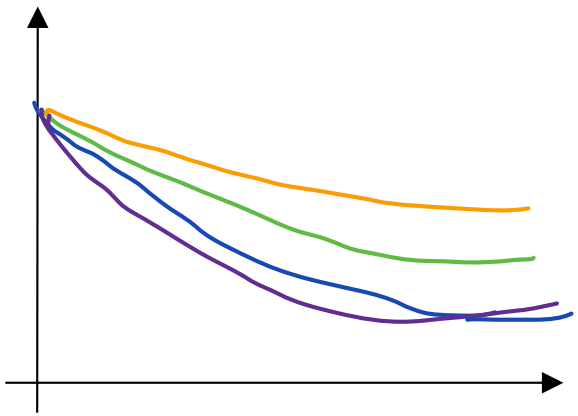
make sure baby panda survives.

Panda

- Each day: increase or decrease the hyperparameters a little (α, β etc.)

Method 2:

Training many models in parallel





Fish strategy: They lay many eggs

Caviar

One egg will do well.

Andrew Ng



deeplearning.ai

Batch Normalization

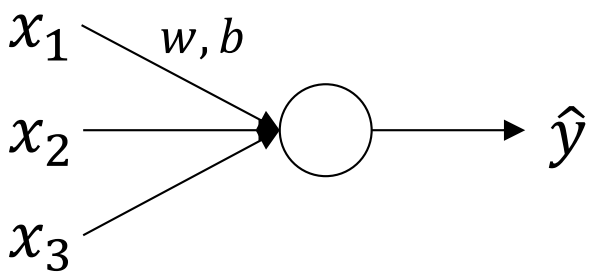
Normalizing activations in a network

- In the rise of deep learning, one of the most important ideas has been an algorithm called batch normalization.

- Batch normalization
  - makes hyperparameter search much easier
  - makes the neural network more robust.
  - the choice of hyperparameters in a much bigger range of hyperparameters work well.
  - will enable training of even very deep networks.

- Next: see how batch normalization works

Normalizing inputs to speed up learning



- In logistic regression normalizing the input can speed up the training process.

$$\mu = \frac{1}{m}\sum_i x^{(i)}, \quad x = x - \mu \quad \text{(Subtract the mean)}$$

$$\sigma^2 = \frac{1}{m}\sum_i x^{(i)2}, \quad x = x / \sigma$$

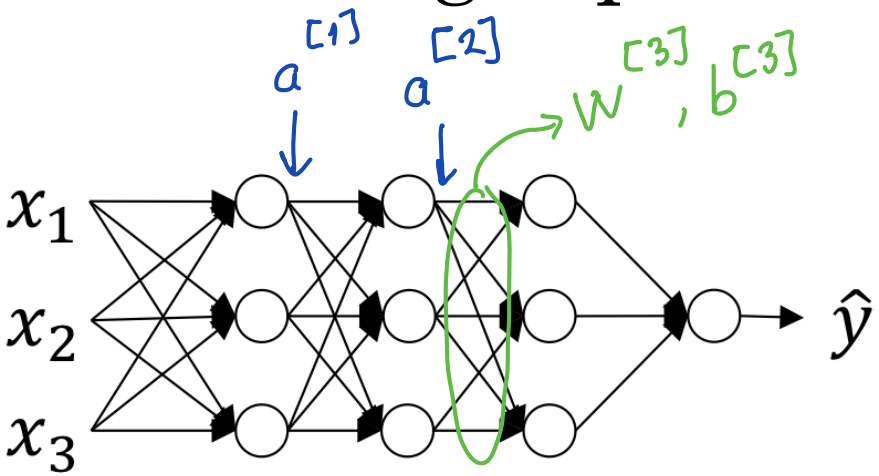
($x^{(i)2}$ → elementwise square)

- This input normalization makes gradient descent based optimization easier, since it makes the contours of the cost function more round instead of elongated.



Andrew Ng

Normalizing inputs to speed up learning



- In a deeper model, we not only have the input but also have activations $a^{[1]}, a^{[2]}$ etc.
- When training the parameters of layer 3 $(W^{[3]}, b^{[3]})$ it would be nice to normalize $a^{[2]}$ to make training of $W^{[3]}, b^{[3]}$ more efficient (faster).
- $a^{[2]}$ is the "input" to layer 3.

  OR Normalize $z^{[2]}$ (done much often)

There's a debate whether to normalize $a^{[2]}$ or $z^{[2]}$.

Andrew Ng

Implementing Batch Norm

Given some intermediate values in NN
(Hidden unit values $z^{(1)}, \ldots, z^{(m)}$ ($z^{[l](i)}$) simplify by omitting [l]

$$\mu = \frac{1}{m} \sum_i z^{(i)}$$ # compute the mean

$$\sigma^2 = \frac{1}{m} \sum_i (z^{(i)} - \mu)^2$$ # compute the variance

$$z^{(i)}_{norm} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$ # normalize to make mean zero and variance 1.

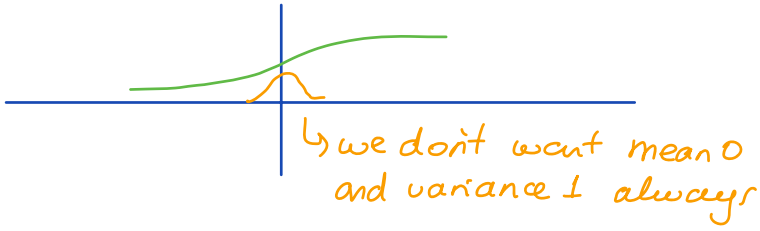
$$\tilde{z}^{(i)} = \gamma z^{(i)}_{norm} + \beta$$ # we don't want the hidden units to always have mean 0 and variance 1. It may make more sense for the hidden units to have a different distribution.

learnable parameters

Use $\tilde{z}^{(i)}$ instead of $z^{(i)}$

- During training update γ and β, like updating the parameters of the network (W etc.)
- γ and β allows to set the mean of $\tilde{z}$ to any value.
  e.g. if $\beta = \mu$ and $\gamma = \sqrt{\sigma^2 + \varepsilon}$ the equation above would be inverted. ($\tilde{z}^{(i)} = z^{(i)}$)

- Summary: Normalization of input helps during optimization of the network.
  - Batch norm applies normalization to normalize the mean and variance of some of hidden units' values.
  - We don't want $z^{(i)}$ to always have mean 0 and variance 1, we want them to have larger variance to take advantage of the nonlinearity of the activation function.



  - The mean and variance of hidden unit values are controlled by γ and β, which are learned.

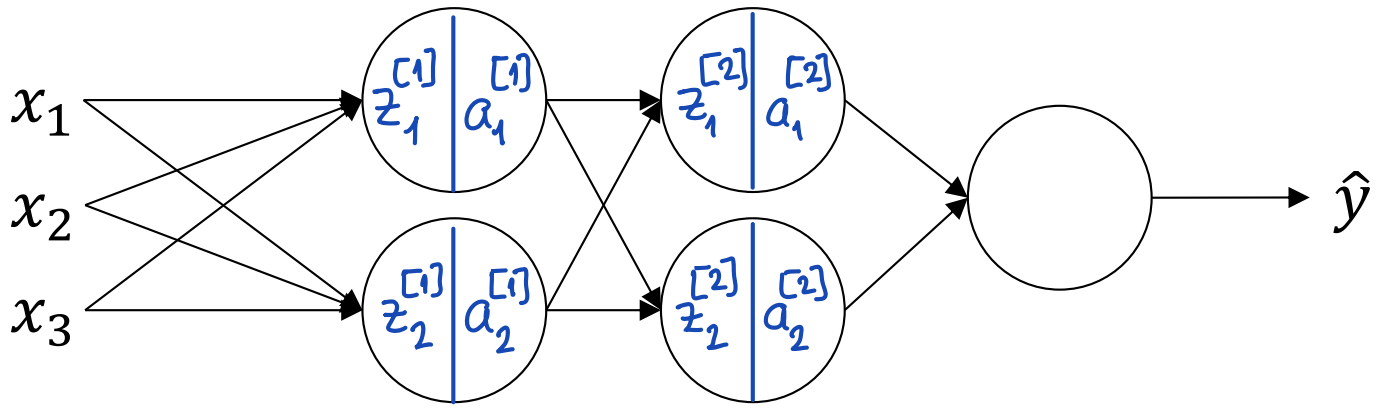


deeplearning.ai

Batch Normalization

Fitting Batch Norm into a neural network

Adding Batch Norm to a network



input X $\xrightarrow[b^{[1]}]{W^{[1]}}$ $z^{[1]}$ $\xrightarrow[\text{BatchNorm (BN)}]{\beta^{[1]}, \gamma^{[1]}}$ $\tilde{z}^{[1]}$ $\rightarrow a^{[1]} = g^{[1]}(\tilde{z}^{[1]})$ $\xrightarrow[b^{[2]}]{W^{[2]}}$ $z^{[2]}$ $\xrightarrow[BN]{\beta^{[2]}, \gamma^{[2]}}$ $\tilde{z}^{[2]}$ $\rightarrow a^{[2]}$

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, \ldots, W^{[L]}, b^{[L]}$

$\beta^{[1]}, \gamma^{[1]}, \beta^{[2]}, \gamma^{[2]}, \ldots, \beta^{[L]}, \gamma^{[L]}$

Compute $d\beta^{[l]}$

$\beta^{[l]} \xrightarrow{\text{update}} \beta^{[l]} = \beta^{[l]} - \alpha\, d\beta^{[l]}$

Tensorflow implementation:

tf.nn.batch-normalization

You can also use momentum, RMSprop, Adam to do the update

Andrew Ng

- In practice, Batch Norm is usually applied with mini-batches of the training set.

Working with mini-batches

$X^{\{1\}} \xrightarrow[b^{[1]}]{W^{[1]}} z^{[1]} \xrightarrow[\text{B.N.}]{\beta^{[1]},\gamma^{[1]}} \tilde{z}^{[1]} \longrightarrow g^{[1]}(\tilde{z}^{[1]}) = a^{[1]} \xrightarrow[b^{[2]}]{W^{[2]}} z^{[2]} \xrightarrow[\text{B.N.}]{\beta^{[2]},\gamma^{[2]}}$

mini batch 1

$X^{\{2\}} \xrightarrow[b^{[1]}]{W^{[1]}} z^{[1]} \xrightarrow[\text{B.N.}]{\beta^{[1]},\gamma^{[1]}} \tilde{z}^{[1]} \longrightarrow g^{[1]}(\tilde{z}^{[1]}) = a^{[1]} \xrightarrow[b^{[2]}]{W^{[2]}} z^{[2]} \longrightarrow \cdots$

mini batch 2

Parameters update: when using Batch Norm, set $b^{[l]}$ to zero:

$W^{[l]}$, ~~$b^{[l]}$~~, $\beta^{[l]}$, $\gamma^{[l]}$

$(n^{[l]}, 1)$ $(n^{[l]}, 1)$ $(n^{[l]}, 1)$

$z^{[l]} : (n^{[l]}, 1)$

originally: $z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$

After mean subtraction in B.N., $b^{[l]}$ will be subtracted, so we don't need it.

$z^{[l]} = W^{[l]} a^{[l-1]}$ is sufficient.

$\tilde{z}^{[l]}_{norm} = \gamma^{[l]} z^{[l]}_{norm} + \beta^{[l]}$

Andrew Ng

Implementing gradient descent

for $t = 1, \ldots,$ numMiniBatches

Compute forward propagation on $X^{\{t\}}$

In each hidden layer, use BN to replace $z^{[l]}$ with $\tilde{z}^{[l]}$

Use backward propagation to compute $dW^{[l]}$, ~~$db^{[l]}$~~, $d\beta^{[l]}$, $d\gamma^{[l]}$

Update parameters:

$$W^{[l]} := W^{[l]} - \alpha\, dW^{[l]}$$

$$\beta^{[l]} := \beta^{[l]} - \alpha\, d\beta^{[l]}$$

$$\gamma^{[l]} := \ldots$$

OR use momentum, RMSprop, Adam

Andrew Ng



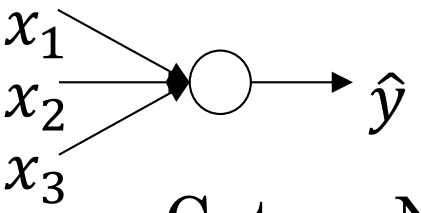
deeplearning.ai

Batch Normalization

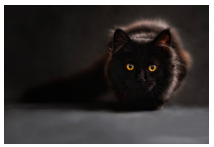
Why does Batch Norm work?

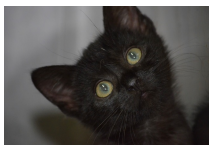
- Why does Batch Norm speed up training?
  - Normalizing input features x to mean zero and variance 1 speeds up learning.
  - Reason 1: Batch norm does a similar thing, for values in the hidden units.
  - Reason 2: It makes weights in a deep layer (e.g. layer 10) more robust to changes to weights in earlier layers of the network (e.g. layer 1).
  - Reason 3: It has a slight regularization effect.

Learning on shifting input distribution



Cat $y = 1$



Non-Cat $y = 0$



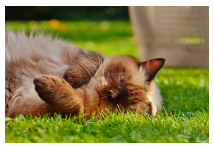



Training Set
(All black cats)

Covariate Shift

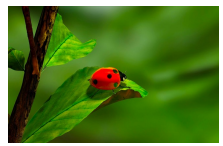
$y = 1$






$y = 0$

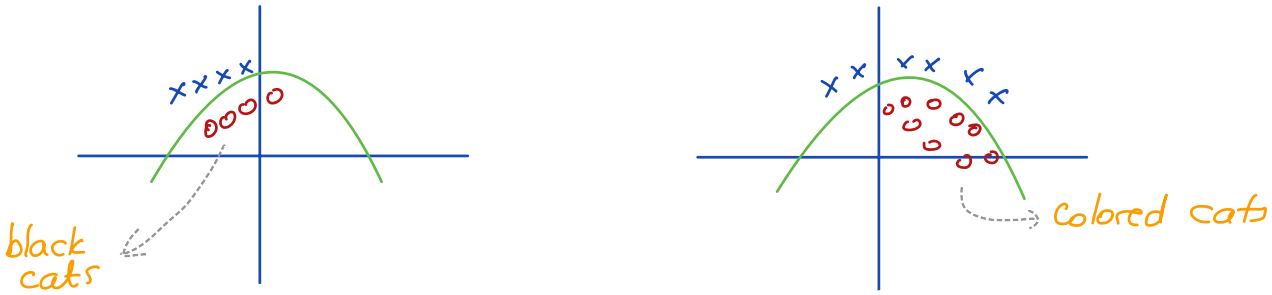





Test Set
(colored cats)

Andrew Ng

- Consider the cat detection problem.
- If you train the deep network using a dataset consisting of all black cats, it may not be successful when you test it on a dataset of colored cats.

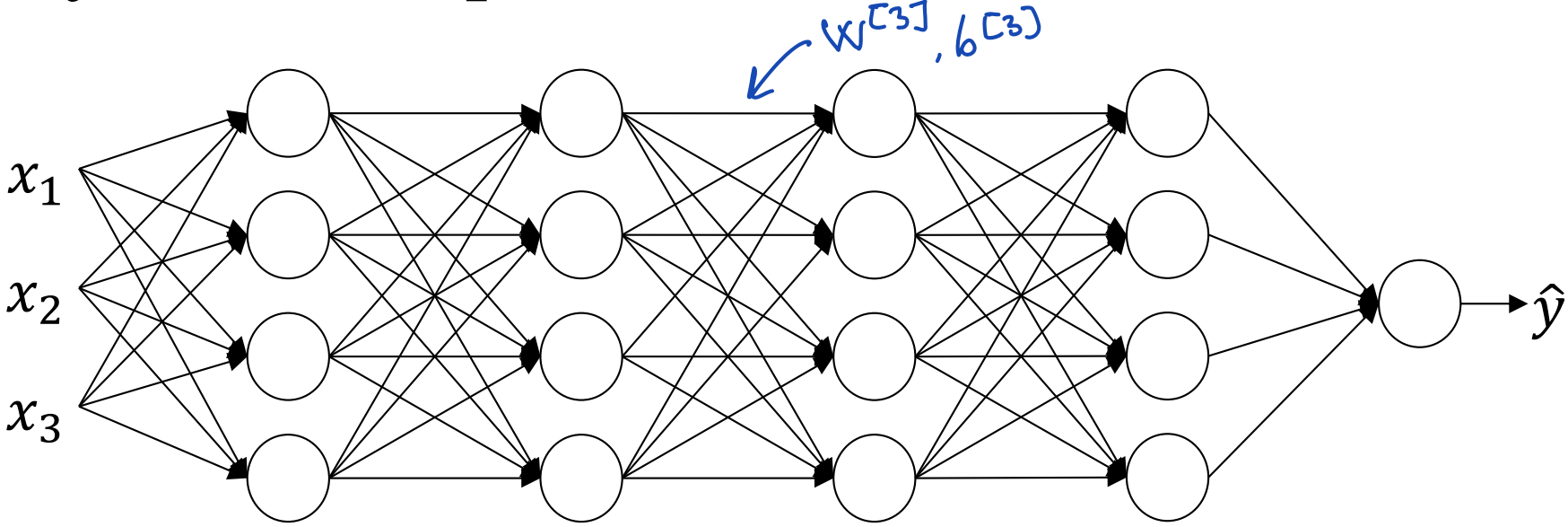


x: negative examples o: positive examples

- Even if the same green decision boundary works well in both cases, we can't expect the learning algorithm to discover the green decision boundary, just by looking at the data on the left.
- Covariate shift: If you've learned a $X \rightarrow Y$ mapping, if the distribution of X changes, then you may need to retain your algorithm. This is true even if the ground truth function (green function) remains unchanged.
    - If the ground truth function changes, the problem of covariate shift is more severe.

- How does this problem of covariate shift apply to a neural network?

Why this is a problem with neural networks?

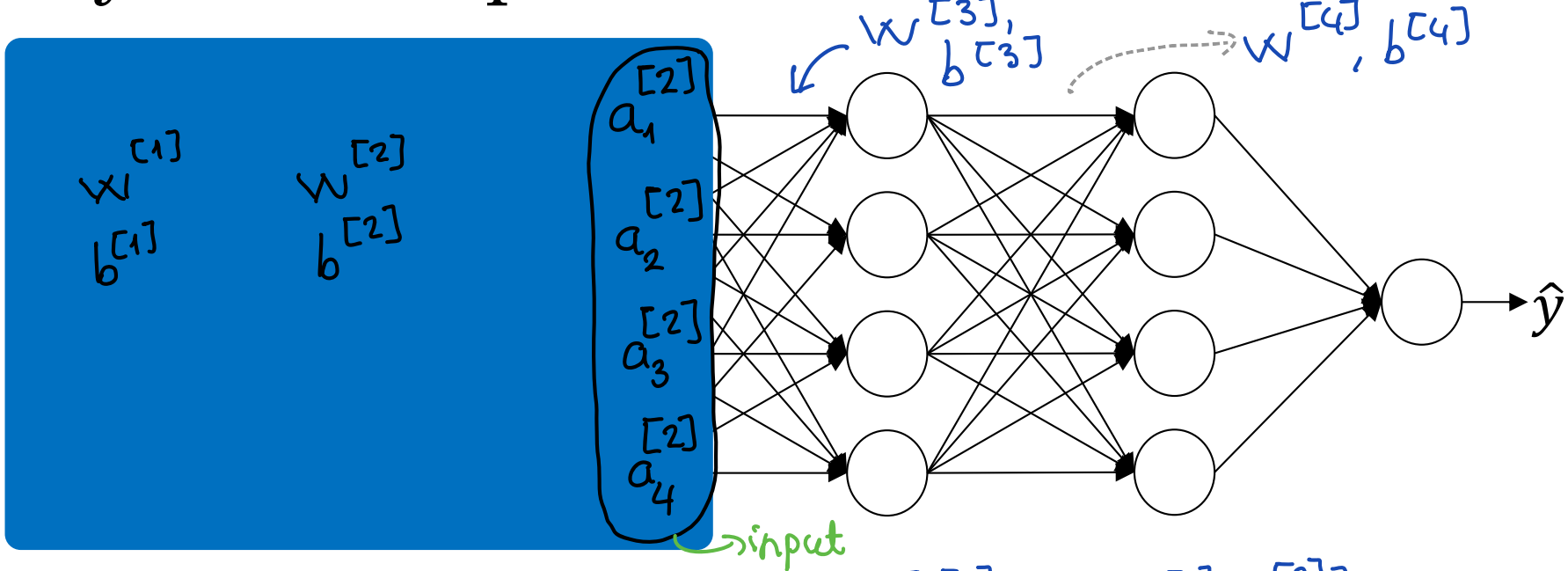


- Consider a deep NN above.
- Look at the learning process from the perspective of the third layer: it gets some values from earlier layers, and it does some operations to make $\hat{y}$ close to y.

Andrew Ng

- Let's cover the left part of the network temporarily.

Why this is a problem with neural networks?

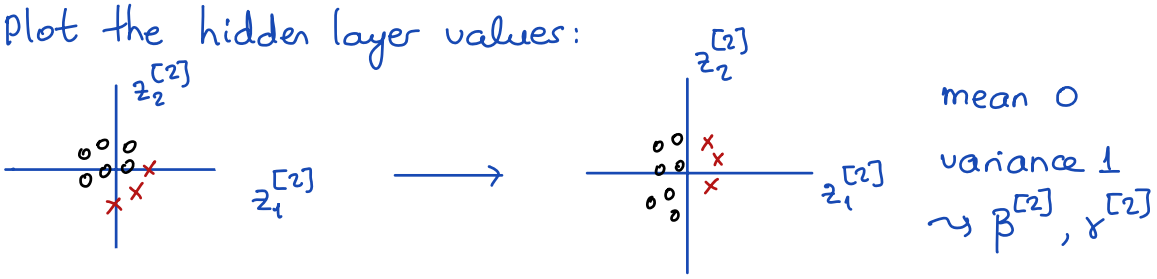


- The input to the third layer are $[a_1^{[2]}, a_2^{[2]}, a_3^{[2]}, a_4^{[2]}]$ and the uncovered layers take this as input and convert to $\hat{y}$.
- But $[a_1^{[2]}, a_2^{[2]}, a_3^{[2]}, a_4^{[2]}]$ are changing as the weights of previous layers ($W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$) change.
- So, this causes a covariate shift problem for layer 3.

Andrew Ng

- What Batch Norm does: it reduces the amount of distribution these hidden unit values shift around.

  Plot the hidden layer values:



- The values of $z_1^{[2]}$ and $z_2^{[2]}$ will change when the network updates the parameters in the earlier layers. But Batch Norm ensures their mean and variance of $z_1^{[2]}$ and $z_2^{[2]}$ will remain the same (e.g. mean 0, variance 1), or whatever value is governed by $\beta^{[2]}$ and $\gamma^{[2]}$.
- It limits the amount to which updating the parameters in the earlier layers can affect the distribution of values that the third layer sees and has to learn on.
- The hidden unit outputs become more stable.
- The next layer learns the parameters independently of other layers and this speeds up learning in the whole network.

Batch Norm as regularization

$X^{\{t\}}$ (64, 128)

- Each mini-batch is scaled by the mean/variance computed on just that mini-batch. (not the whole dataset)
- This adds some noise to the values $z^{[l]}$ within that minibatch. So similar to dropout, it adds some noise to each hidden layer's activations.
- This has a slight regularization effect.

- Dropout has multiplicative noise because it multiplies by 0 or 1. NN doesn't rely on a single weight.
- BatchNorm also has multiplicative noise since it is scaling by σ, as well as additive noise since mean μ is subtracted.

Andrew Ng

- This has a slight regularization effect.
- BatchNorm could be used together with Dropout to have a strong regularization effect.
- If you increase the mini-batch size from 128 to 512, the noise, and hence the regularization effect is reduced.
- Regularization is an unintended side effect of BatchNorm.



deeplearning.ai

Batch Normalization

Batch Norm at test time

- During training, BatchNorm handles data one mini-batch at a time. It computes means and variances on mini-batches.
- At test time we may process one sample at a time.
- So, at test time we need to do something slightly differently to make sure the predictions make sense.
- μ and σ of 1 example doesn't make sense.

Batch Norm at test time

Training-Time: m: # samples in mini-batch

$$\mu = \frac{1}{m}\sum_i z^{(i)}$$

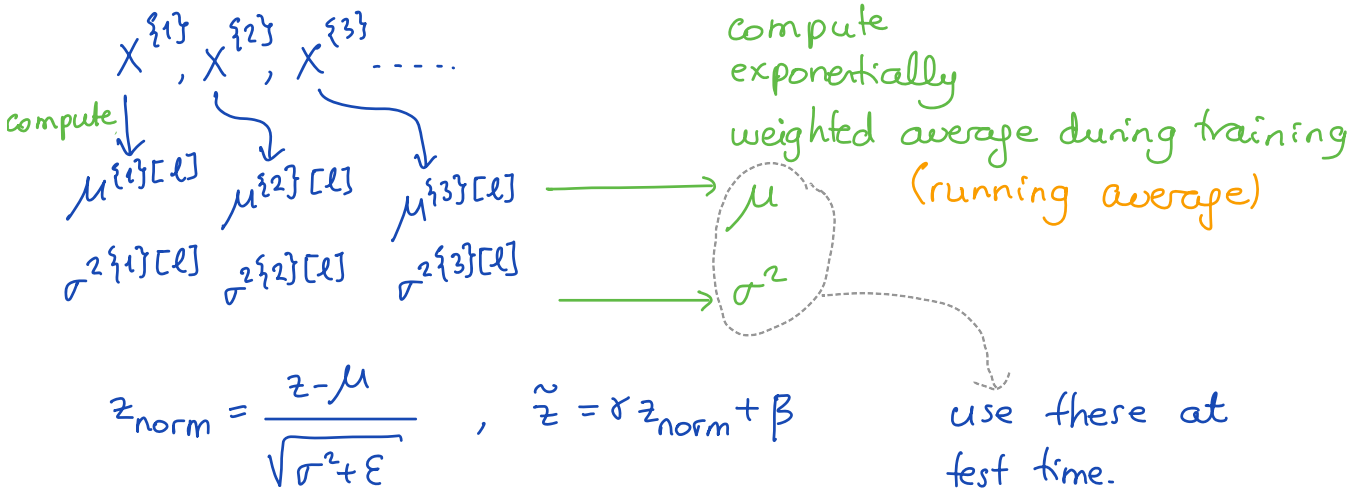
$$\sigma^2 = \frac{1}{m}\sum_i (z^{(i)} - \mu)^2$$

$$z^{(i)}_{\text{norm}} = \frac{z^{(i)} - \mu}{\sqrt{\sigma^2 + \varepsilon}}$$

$$\tilde{z}^{(i)} = \gamma z^{(i)}_{\text{norm}} + \beta$$

μ, σ^2: estimate using exponentially weighted averages (across mini-batches)

- pick a layer $[l]$ and we are going over mini-batches.



$$z_{norm} = \frac{z - \mu}{\sqrt{\sigma^2 + \varepsilon}} \quad , \quad \tilde{z} = \gamma z_{norm} + \beta$$

use these at test time.



deeplearning.ai

Multi-class classification

Softmax regression

class labels

0: other

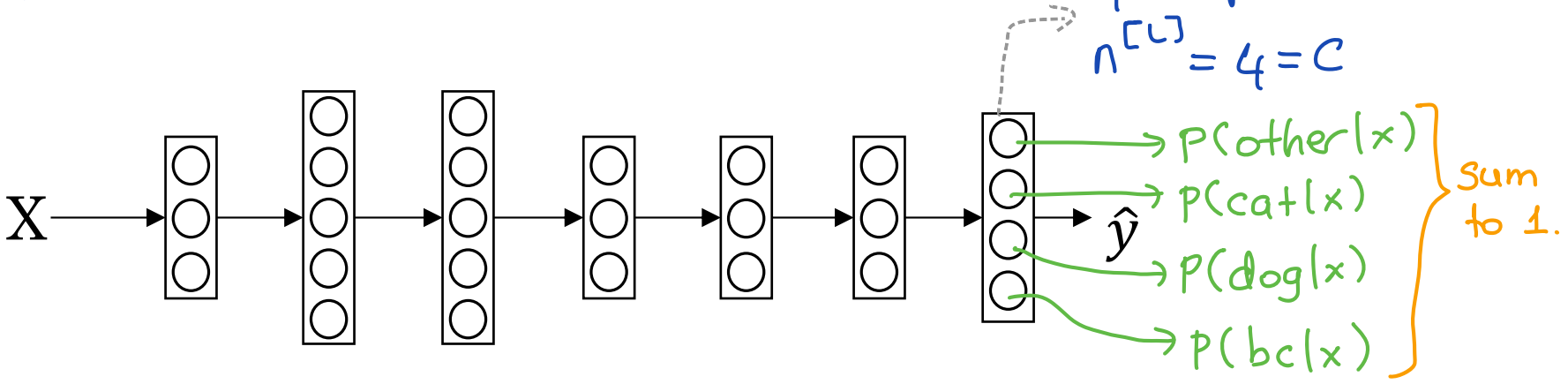
1 2 3

Recognizing cats, dogs, and baby chicks: 3 classes



3 1 2 0 3 2 0 1

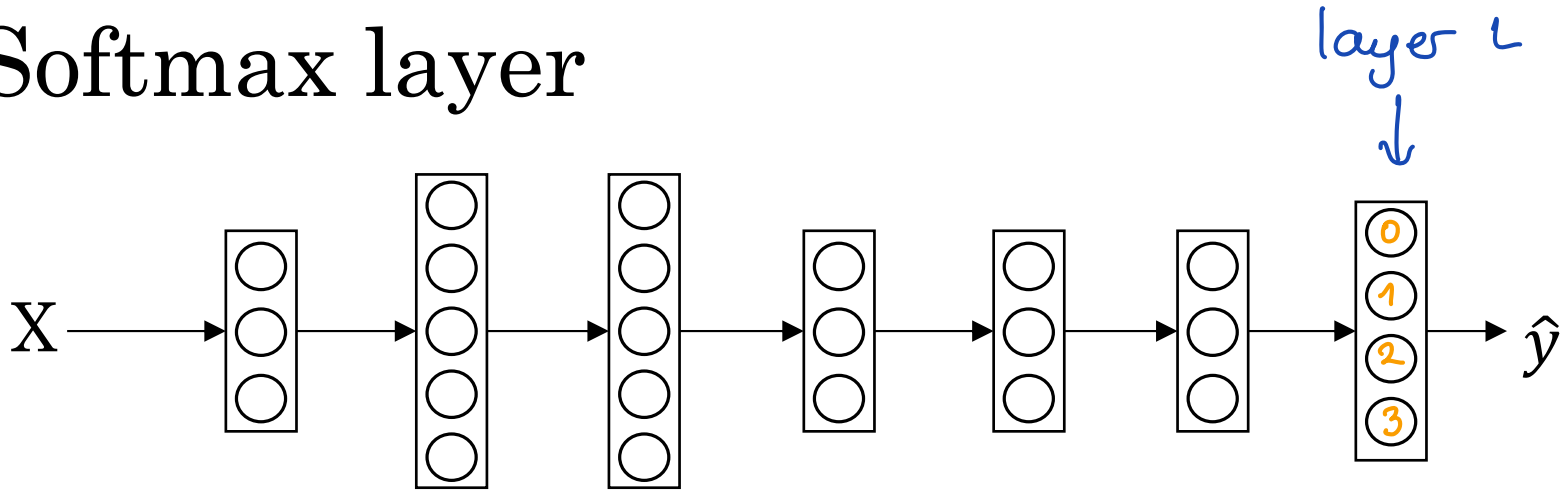
C = # classes = 4 (0, 1, 2, 3)



$\hat{y}$ is (4,1)

Andrew Ng

Softmax layer



$z^{[L]} = W^{[L]} a^{[L-1]} + b^{[L]}$

elementwise

Activation function: $t = e^{(z^{[L]})}$ (4,1)

$$a^{[L]} = \frac{e^{(z^{[L]})}}{\sum_{i=1}^{4} t_i}, \quad a_i^{[L]} = \frac{t_i}{\sum_{i=1}^{4} t_i}$$

(4,1)

i^{th} element

$a^{[L]} = g^{[L]}(z^{[L]})$

(4,1) vector

↳ (4,1) : vector

Andrew Ng

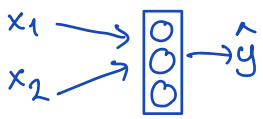
e.g. $z^{[l]} = \begin{bmatrix} 5 \\ 2 \\ -1 \\ 3 \end{bmatrix} \rightarrow t = \begin{bmatrix} e^5 \\ e^2 \\ e^{-1} \\ e^3 \end{bmatrix} = \begin{bmatrix} 148.4 \\ 7.4 \\ 0.4 \\ 20.1 \end{bmatrix}$, $\sum_{i=1}^{4} t_i = 176.3$

all positive

$$a^{[l]} = \frac{t}{176.3} = \begin{bmatrix} e^5/176.3 \\ e^2/176.3 \\ e^{-1}/176.3 \\ e^3/176.3 \end{bmatrix} = \begin{bmatrix} 0.842 \\ 0.042 \\ 0,002 \\ 0,114 \end{bmatrix}$$

→ class 0 has the highest probability

EX Consider a NN with 2 inputs, no hidden layer and a softmax output layer.

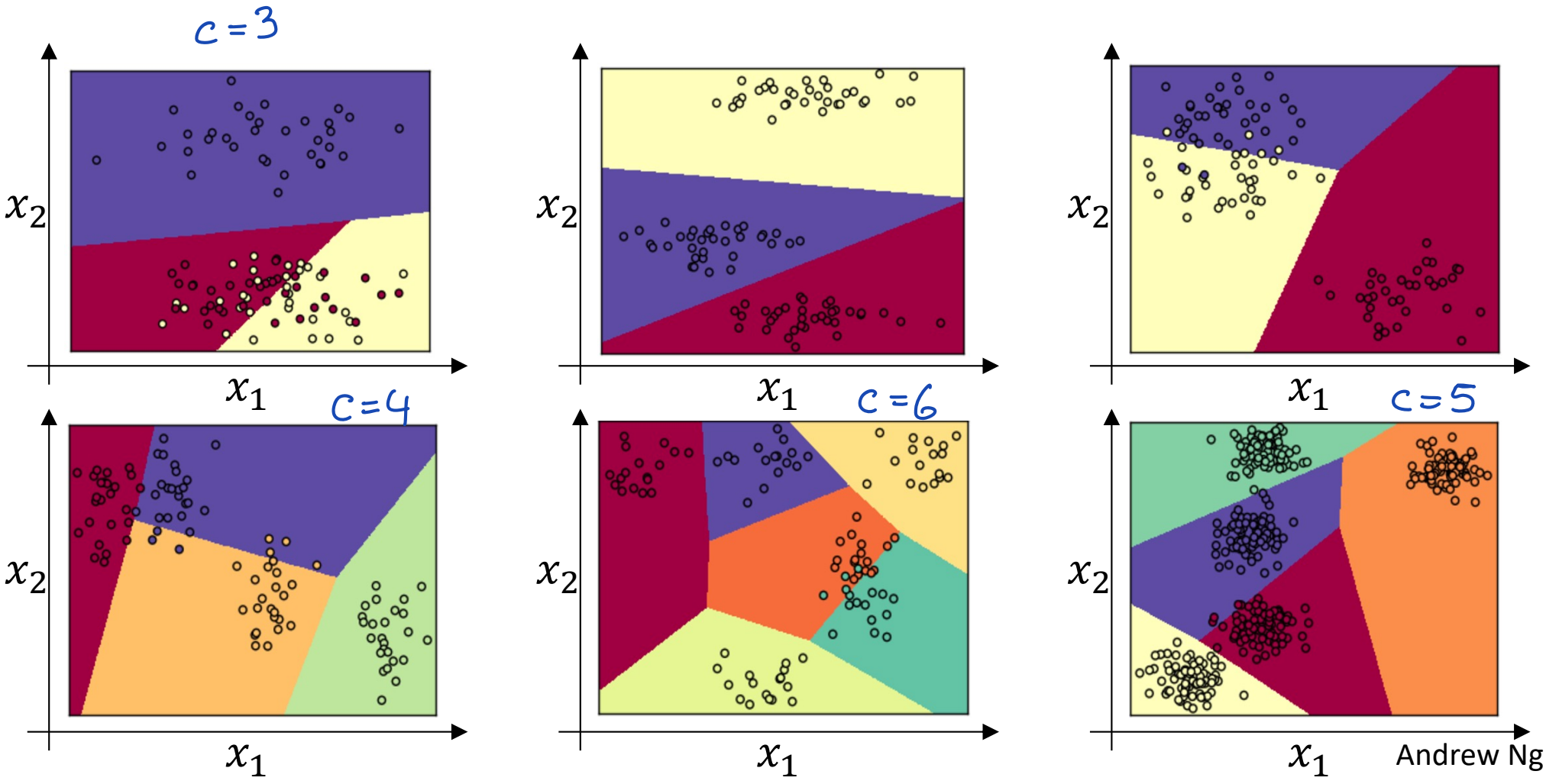


$$z^{[1]} = W^{[1]} x + b^{[1]}$$
$$a^{[1]} = \hat{y} = g(z^{[1]})$$

↳ softmax activation

- Generalization of logistic regression with more than 2 classes.
- Decision boundaries are linear between the classes.

Softmax examples



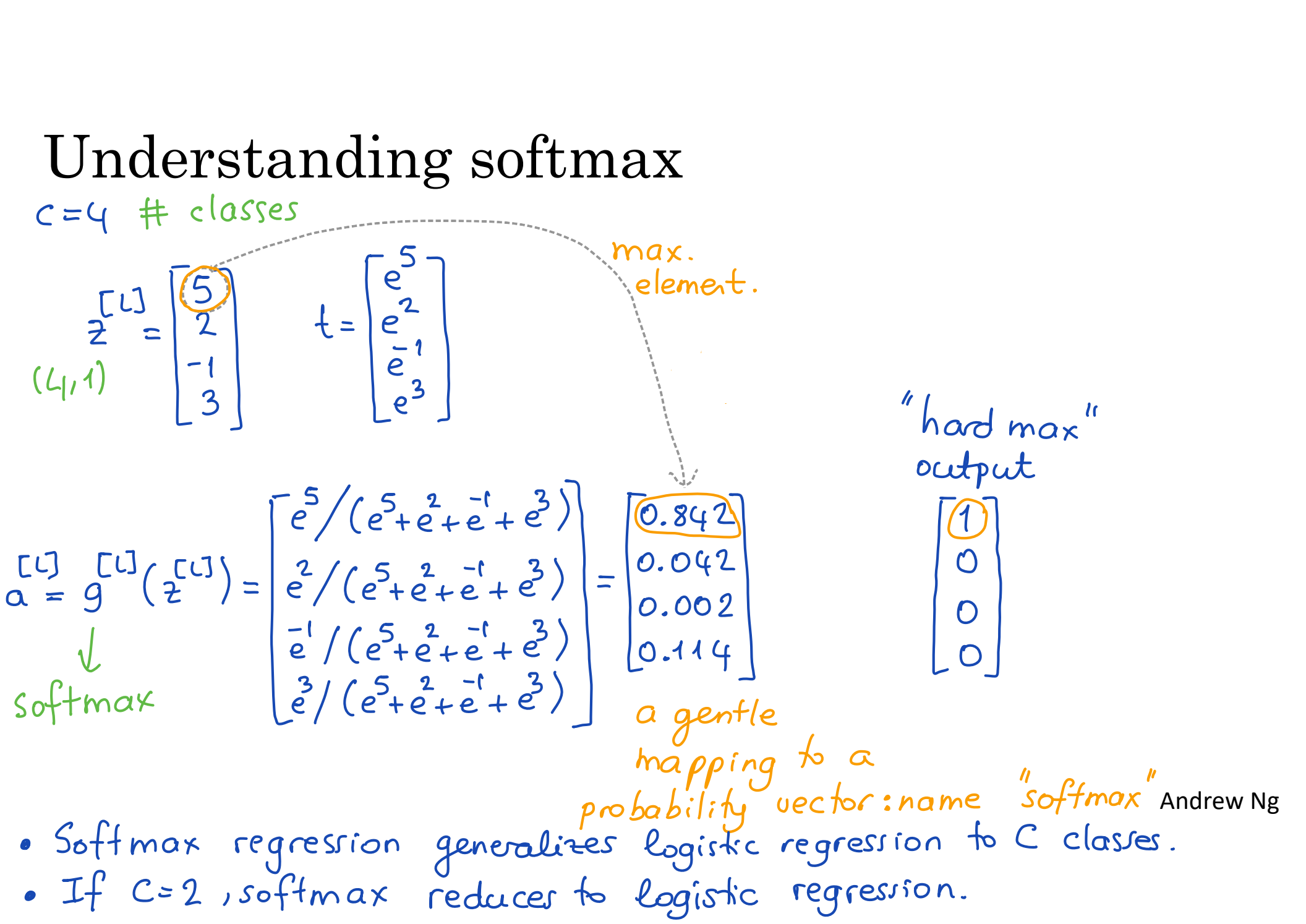


deeplearning.ai

Multi-class classification

Training a softmax classifier

Understanding softmax



- Softmax regression generalizes logistic regression to C classes.
- If C=2, softmax reduces to logistic regression.

Andrew Ng

Loss function

Target output:

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \rightsquigarrow \text{cat} \qquad (4,1)$$

Estimated output:

$$a^{[L]} = \hat{y} = \begin{bmatrix} 0.3 \\ 0.2 \\ 0.1 \\ 0.4 \end{bmatrix} \rightsquigarrow \text{not very high} \qquad (4,1)$$

Loss function:

$$\mathcal{L}(\hat{y}, y) = -\sum_{j=1}^{4} y_j \log \hat{y}_j \qquad \text{(cross-entropy loss)}$$

(note that $y_1 = y_3 = y_4 = 0$)

$$= -y_2 \log \hat{y}_2 = -\log \hat{y}_2$$

- To make $\mathcal{L}(\hat{y}, y)$ small, $-\log \hat{y}_2$ should be small, hence $\hat{y}_2$ should be big.

Andrew Ng

- Cost function for all samples:

$$J(w^{[1]}, b^{[1]}, \ldots) = \frac{1}{m} \sum_{i=1}^{n} \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

- In vectorized notation:

$$Y = \begin{bmatrix} y^{(1)} & y^{(2)} & \cdots & y^{(m)} \end{bmatrix}$$

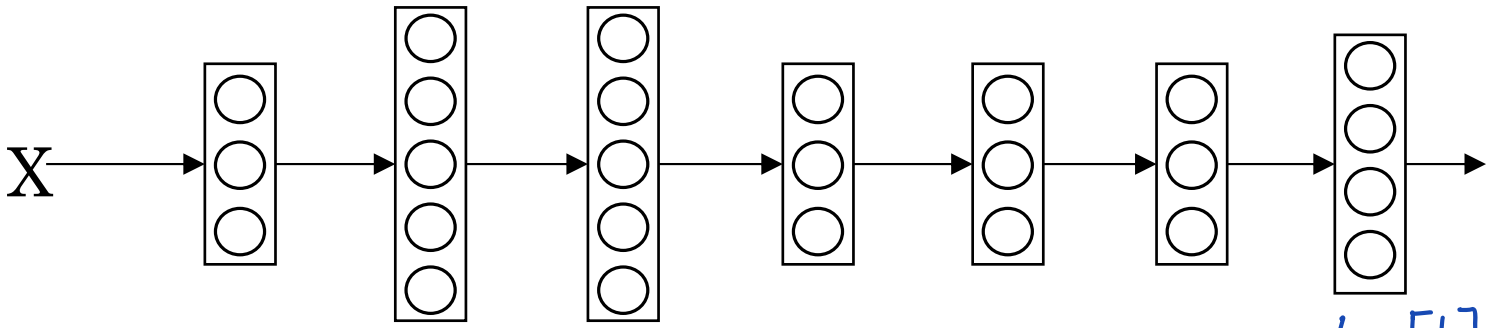
$$= \begin{bmatrix} 0 & 0 & 1 & \cdots \\ 1 & 0 & 0 & \\ 0 & 1 & 0 & \\ 0 & 0 & 0 & \end{bmatrix}_{(4,m)}$$

$$\hat{Y} = \begin{bmatrix} \hat{y}^{(1)}, \ldots, \hat{y}^{(m)} \end{bmatrix}$$

$$= \begin{bmatrix} 0.3 & \cdots \\ 0.2 & \\ 0.1 & \\ 0.4 & \end{bmatrix}_{(4,m)}$$

Gradient Descent with Softmax

Summary of softmax classifier



Forward Pass:

$\rightarrow z^{[L]} \rightarrow a^{[L]} = \hat{y} \rightarrow \mathcal{L}(\hat{y}, y)$

$(4,1)$

Backpropagation: $dz^{[L]} = \hat{y} - y$

$(4,1)$

$$\frac{\partial J}{\partial z^{[L]}}$$

- In a deep learning programming framework, get the forward pass right and the backward pass will be done by the framework.

Andrew Ng